

- Hello everyone, this is Mohammad Abdulaziz, a lecturer at the dept of informatics
- I do research in interactive theorem proving and its applications to making sure that AI systems and algorithms are safe (I will say a bit more about that)
- I will be teaching you three lectures on interactive theorem proving, an important technique used for formal verification and other things

## Interactive Theorem Proving: Introduction

Formal Verification: 6CCS3VER

## This Video

- What is interactive theorem proving?
- What are we doing over the next three lectures?
- What do I expect you to already know?
- Some Organisational notes

In this video I will

- Introduce you to interactive theorem proving, the main idea, and motivate why I think it is an interesting technology to learn about
- I will then discuss briefly what are we doing over the next three lectures, which talk about ITPing, including the learning objectives
- I will also briefly discuss what concepts and techniques I expect you to already know from previous modules, although I will briefly introduce the different concepts throughout the lectures
- I will discuss some organisational notes regarding these next lectures and the coursework

# What is ITP'ing?

## Formal proofs:

- Recall with me the following Aristotlian syllogism
  - 1: All men are mortal (Assume)
  - 2: Aristotle is a man (Assume)
  - 3: Aristotle is mortal (from 1, 2, and modus-ponens)
- Modus-ponens: if a implies b and a holds, then b holds
- I hope we all agree that this is an unambiguously correct proof

- Before I talk about interactive theorem proving I want to recapitulate with you the concept of formal proof
- Recall with me the following Aristotlian syllogism
  - All men are mortal (Assume)
  - Aristotle is a man (Assume)
  - Aristotle is mortal (from 1, 2, and modus-ponens)
- I hope we all agree that this is an unambiguously correct proof

# What is ITP'ing?

- Our objective: construct formal proofs
- Unambiguous, correct proofs:
  - Definitions' and theorem statements' meaning and well-formedness
  - Proof correctness
- All of that wrt precisely and concisely stated axioms

- Our goal is to construct formal proofs of correctness for mathematical theorems and for algorithms/systems
- This is because formal proofs are the closest thing to objectively and unambiguously correct proofs:
  - By that I mean, definitions and theorem statements whose meaning and well-formedness are unambiguously correct
  - And also proof that are unambiguously correct
  - This is to be done wrt precisely and concisely stated axioms that indicate what are correct reasoning or proof steps, and what is the meaning of different statements or claims

## What is ITP'ing?

- Proofs and definitions are written in a formal language
- Meaning is defined by set of axioms, e.g.
  - Linear temporal logic axioms (recall from week 3 lecture)
  - More general: Zermelo-Fraenkel set theory:
    - \* Extensionality:  $\forall s, t. (\forall x. x \in s \leftrightarrow x \in t) \rightarrow s = t$
    - \* Regularity:  $\forall s. s \neq \emptyset \rightarrow \exists t. t \in s \wedge t \cap s = \emptyset$
    - \* Schema:  $\forall s, x_1, \dots, x_n. \exists t. \forall y. y \in t \leftrightarrow (y \in s \wedge \phi(s, x_1, \dots, x_n, y))$
    - \* ...
  - Others formalisms
    - \* Type theory, Peano Arithmetic, First-order logic, and Higher-order logic (HOL)

- Proofs and definitions are written in formal language whose meaning is defined by set of axioms, e.g.
  - Linear temporal logic axioms (recall from week 3 lecture)
  - or Zermelo-Fraenkel set theory that are capable of representing any mathematical statement or thm, and they are more general than LTL. Some of its axioms are as follows
    - \* Extensionality:  $\forall s, t. (\forall x. x \in s \leftrightarrow x \in t) \rightarrow s = t$
    - \* Regularity:  $\forall s. s \neq \emptyset \rightarrow \exists t. t \in s \wedge t \cap s = \emptyset$
    - \* Schema:  $\forall s, x_1, \dots, x_n. \exists t. \forall y. y \in t \leftrightarrow (y \in s \wedge \phi(s, x_1, \dots, x_n, y))$
    - \* ...
  - there is also Type theory
  - Peano Arithmetic
  - First-order logic
  - and Higher-order logic (HOL), which is the logical system of the theorem prover we use here

# What is ITP'ing?

- Why formal proofs?
- Initially to “formalise” mathematics
  - E.g. Euclid's elements ( 300 B.C.) and principia mathematica [Russel and Whitehead 1910]

- The initial reason to bother with formal proofs, which is still very relevant (later on that), is to “formalise” mathematics, which was more of an intellectual curiosity
  - Some of the earliest trials to get formal proofs, i.e. to fully specify the axioms and show how every step in the proof follows from those axioms, are Euclid's elements ( 300 B.C.) and principia mathematica [Russel and Whitehead 1910]

# What is ITP'ing?

- Challenge: is the size of a formal proof
  - Russel and Whitehead prove that  $1+1=2$  after 173 pages

$$*54.43. \quad \vdash :. \alpha, \beta \in 1 . \supset : \alpha \cap \beta = \Lambda . \equiv . \alpha \cup \beta \in 2$$

*Dem.*

$$\begin{aligned} &\vdash . *54.26 . \supset \vdash :. \alpha = \iota'x . \beta = \iota'y . \supset : \alpha \cup \beta \in 2 . \equiv . x \neq y . \\ &[*51.231] \qquad \qquad \qquad \equiv . \iota'x \cap \iota'y = \Lambda . \\ &[*13.12] \qquad \qquad \qquad \equiv . \alpha \cap \beta = \Lambda \qquad (1) \end{aligned}$$

$$\begin{aligned} &\vdash . (1) . *11.11.35 . \supset \\ &\quad \vdash :. (\exists x, y) . \alpha = \iota'x . \beta = \iota'y . \supset : \alpha \cup \beta \in 2 . \equiv . \alpha \cap \beta = \Lambda \qquad (2) \end{aligned}$$

$\vdash . (2) . *11.54 . *52.1 . \supset \vdash . \text{Prop}$   
 From this proposition it will follow, when arithmetical addition has been defined, that  $1 + 1 = 2$ .

- Unrealistic for non-trivial proofs, due to size and tedium
- Never took-off, except as a theoretical idea

- A challenge, however, to that way of doing mathematics is size of a formal proof
  - An example from Russel and Whitehead's attempt is shown on the slide, where in their book on formal mathematics they were able to prove that  $1+1=2$  after 173 pages of definitions and proofs
  - Because of that, it was seen as unrealistic for any non-trivial proofs, due to size and tedium, and
  - it never took-off, except as a theoretical idea

$$*54.43. \quad \vdash :. \alpha, \beta \in 1 . \supset : \alpha \cap \beta = \Lambda . \equiv . \alpha \cup \beta \in 2$$

*Dem.*

$$\begin{aligned} &\vdash . *54.26 . \supset \vdash :. \alpha = \iota'x . \beta = \iota'y . \supset : \alpha \cup \beta \in 2 . \equiv . x \neq y . \\ &[*51.231] \qquad \qquad \qquad \equiv . \iota'x \cap \iota'y = \Lambda . \\ &[*13.12] \qquad \qquad \qquad \equiv . \alpha \cap \beta = \Lambda \qquad (1) \end{aligned}$$

$$\begin{aligned} &\vdash . (1) . *11.11.35 . \supset \\ &\quad \vdash :. (\exists x, y) . \alpha = \iota'x . \beta = \iota'y . \supset : \alpha \cup \beta \in 2 . \equiv . \alpha \cap \beta = \Lambda \qquad (2) \end{aligned}$$

$\vdash . (2) . *11.54 . *52.1 . \supset \vdash . \text{Prop}$   
 From this proposition it will follow, when arithmetical addition has been defined, that  $1 + 1 = 2$ .

# What is ITP'ing?

- Machines can check proofs, and generate them
  - Automated theorem provers
    - \* model-checkers
    - \* SAT-solvers
    - \* first-order logic automatic theorem provers
  - Problem: does not scale
  - Interactive theorem provers (ITPs)
    - \* e.g. Isabelle/HOL [Nipkow et al. 2002], HOL4 [Slind and Norrish 2008], Coq [Bertot and Casteran], Lean [de Moura]

- A turning point happened to formal mathematics with the advent of digital computers
- Since computers can be very effective at generating and checking these very long formal proofs
  - One approach to computer-based formal maths is that of automated theorem provers, where the computer generates a formal proof and checks that it is correct according to the logic's axioms.
  - Examples of this approach are model-checkers, with which you must be very well acquainted, and also SAT-solvers, which you will learn about during the last lecture, and also first-order logic ATPs
    - \* model-checkers
    - \* SAT-solvers
    - \* first-order logic automatic theorem provers
  - Although these automated methods have the advantage that they are push-button, they have the problem that they does not scale to bigger or more conceptually complicated mathematics
  - Fundamental reasons for that are Gödel's incompleteness theorem, undecidability of the halting problem
  - Practical implications include for instance that one cannot prove that an algorithm is correct for any input automatically for any given algorithm, or to automatically prove different mathematical theorems about primes, etc.
  - However, a practically very successful compromise is the approach of interactive theorem provers (ITPs) like
    - \* Isabelle/HOL [Nipkow et al. 2002], HOL4 [Slind and Norrish 2008], Coq [Bertot and Casteran], Lean
  - ITPs are mechanised formal mathematical systems
  - in which one specifies definitions, statements, and proofs in a formal language
  - and the ITP makes sure that the proof only follows from a specific set of axioms
  - in our case, we use the ITP Isabelle/HOL
  - An example proof using that system is on the slide, where a proof of the fact that  $\sqrt{2}$  is irrational is presented.
  - As shown here, the user enters high-level steps of the proof, and the ITP fills in the details to make sure that the proof is valid down to the level of logical axioms



# What is ITP'ing?

```
| 1| theory Sqrt2
| 2| imports Main
| 3| begin
| 4| theorem sqrt2_not_rational:
| 5|   "sqrt (real 2)  $\notin$  Q"
| 6| proof
| 7|   let ?x = "sqrt (real 2)"
| 8|   assume "?x  $\in$  Q"
| 9|   then obtain m n :: nat where
|10|     sqrt_rat: "?x = real m / real n" and lowest_terms: "coprime m n"
|11|     by (rule Rats_abs_nat_div_natE)
|12|   hence "real (m^2) = ?x^2 * real (n^2)" by (auto simp add: power2_eq_square)
|13|   hence eq: "m^2 = 2 * n^2" using of_nat_eq_iff power2_eq_square by fastforce
|14|   hence "2 dvd m^2" by simp
|15|   hence "2 dvd m" by simp
|16|   have "2 dvd n" proof
|17|     from 2 dvd m obtain k where "m = 2 * k"
|18|     with eq have "2 * n^2 = 2^2 * k^2" by simp
|19|     hence "2 dvd n^2" by simp
|20|     thus "2 dvd n" by simp
|21|   qed
|22|   with 2 dvd m have "2 dvd gcd m n" by (rule gcd_greatest)
|23|   with lowest_terms have "2 dvd 1" by simp
|24|   thus False using odd_one by blast
|25| qed
|26| end
|27|
|28|
```

- ITPs are mechanised formal mathematical systems
- in which one specifies definitions, statements, and proofs in a formal language
- and the ITP makes sure that the proof only follows from a specific set of axioms
- in our case, we use the ITP Isabelle/HOL
- An example proof using that system is on the slide, where a proof of the fact that sqrt 2 is irrational is presented.
- As shown here, the user enters high-level steps of the proof, and the ITP fills in the details to make sure that the proof is valid down to the level of logical axioms

# What is ITP'ing?

- Notable applications in formal mathematics
  - four-colour theorem [Gonthier 2007]
  - odd-order theorem [Gonthier et al. 2013]
  - Kepler's conjecture [Hales et al. 2015]
  - 98% of the “ most interesting” 100 theorems
  - The Liquid-Tensor Experiment

- ITPs are more successful for practising formal proof, in both maths and computer science
- Notable applications to formal mathematics
  - four-colour theorem [Gonthier 2007]
  - odd-order theorem [Gonthier et al. 2013]
  - Kepler's conjecture [Hales et al. 2015]
  - There is also a list of the “ most interesting” 100 theorems, which keeps tracks of which theorems were formally proved
  - Also recently proof assistants were used to formally prove a theorem whose correctness was doubted in the area of number theory

## What is ITP'ing?

- In CS formal proof is used for *formal verification*:
  - A system and its desired properties are specified in a formal language
- Notable application in CS:
  - To get reliable implementations
    - \* SeL4: Linux Kernel [Klein et al. 2009]
    - \* CompCert: compiler for C [Leroy 2009]
    - \* CakeML: compiler for a flavour of ML [Kumar et al. 2014]
    - \* model-checker [Esparza et al. 2013]
    - \* SAT-solver [Blanchette et al. 2018]
- Adopted by industry:
  - Intel, Arm, AWS, Apple

In CS formal proof is used for *formal verification*:

- A system and its desired properties are specified in a formal language

Notable application in CS:

- To get reliable implementations of complicated algorithms and software. This includes
  - SeL4: Linux Kernel [Klein et al. 2009]
  - CompCert: compiler for C [Leroy 2009]
  - CakeML: compiler for a flavour of ML [Kumar et al. 2014]
  - and even a fully verified model-checker [Esparza et al. 2013], i.e. if it tells a specification has no counter examples, then it definitely doesn't
  - SAT-solver [Blanchette et al. 2018, Fleury 2019]

It is also being more and more adopted by industry in the area of critical software formal verification, e.g. cryptographic software in Apple, many parts of AWS infrastructure, as well as hardware and firmware in Arm and Intel

I have added some links to the job openings related to theorem proving at these companies, which of course will change with time

## What is ITP'ing?

- Some of my own **research**:
  - verification of AI and ML
  - formalisation of mathematics, esp. complexity theory and combinatorial optimisation
- If that interests any of you for a final year project, let's have a chat!

- Finally, I would like to note that a lot of my own work is in the application of ITPs to
  - the verification of algorithms and software in the areas of automated planning, solving MDPs, ML, and reinforcement learning
  - formalisation of mathematics, especially complexity theory and the theory of combinatorial optimisation
- If that interests any of you for a final year project, let's have a chat!

## What will we do (i.e. learning outcomes)?

- In the next three lectures:
  - You will formally prove properties of simple functional programs
  - You will do this pen-and-paper and using the ITP Isabelle/HOL
- Take your own notes from the videos!

- In the next three lectures I will introduce you formal proofs in the context of the interactive theorem prover Isabelle/HOL
- I will focus on how to prove properties of very simple functional programs
- I will also try to show you the process of how you think about or design a formal proof, which is why the rest of these lectures will be me showing you how to find such proofs on a notebook and then how you implement such proofs in Isabelle/HOL
- Hence, there will not be any more slides, and I recommend that you take your own notes based on the videos
- I also recommend that in the coming videos, you try to solve the problems that I solve on your own, i.e. pause the video and try to come up with the ideas

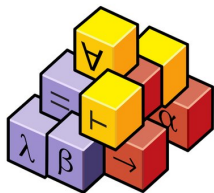
## Expected Background Knowledge

- Mathematical induction (e.g. passed FC1 or FC2)
- First-order logic and how to formulate statements in it (e.g. ELA)
- Basic functional programming (e.g. passed PEP, esp. the Scala part)
- I will refresh them!

Although I will briefly re-introduce you to the following concepts, I expect you to have some familiarity with

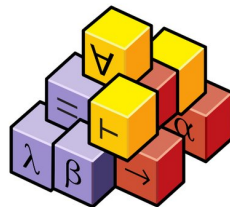
- Mathematical induction (e.g. passed FC1 or FC2)
- Familiarity with first-order logic and how to formulate statements in it (e.g. passed ELA)
- Familiarity with functional programming (e.g. passed PEP, esp. the Scala part)

## Organisational Points



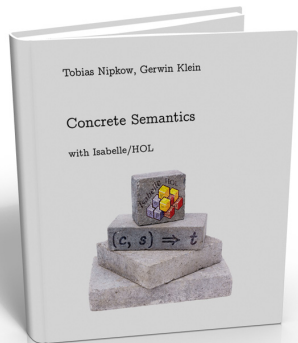
- We will use the **Isabelle/HOL** theorem prover
  - Developed by Cambridge, TU Munich, UNSW, and many others
- Questions: KEATS+**Isabelle's Zulip channel**

- As I mentioned earlier, we will be using the **Isabelle/HOL** theorem prover
- It was initially developed by Larry Paulson, from the university of Cambridge, and it is currently maintained by people from the University of Cambridge, TU Munich, and many others
- Please download it from the link provided on this slide
- In addition to asking me or on KEATS, I recommend you sign up to **Isabelle's Zulip channel** and submit your questions there, you will find many people in the Isabelle community who are willing to help.

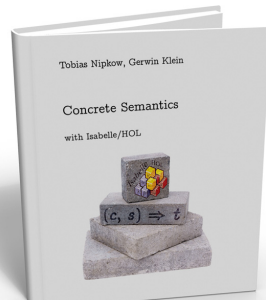


# Organisational Points

- First half of **Concrete Semantics**
- A brief Isabelle/HOL documentation: **Programming and Proving**



- Finally, I would like to note a few relevant points
- The first regards books.
- I think the best book would be **Concrete Semantics**, especially the first half of the book.
- It gives a good introduction to Isabelle and to functional programming and to proving theorems about those functional programs.
- There is also a brief documentation which comes with Isabelle/HOL as you install it, and it is called **Programming and Proving**.
- This will have a lot of information on available proof methods in Isabelle, etc.





## Organisational Points

- Assessment:
  - Coursework: an Isabelle assignment, deadline on 13th December
  - Exam questions: pen-and-paper proofs of properties of a functional program
- The assessment of this part will be divided between two things:
  - Coursework: this will be an Isabelle assignment, with deadline on 13th December
  - Exam questions: this will be pen-and-paper proofs of properties of simple functional programs